

1. Abstract

CINE-IQ is a full-stack, production-ready movie recommendation system built around a central design principle: every recommendation must be explainable. Modern streaming platforms suffer from a "black box" problem — users receive suggestions with no insight into why. CINE-IQ addresses this by combining three classical machine learning techniques into a weighted hybrid ensemble, layering a sentiment re-ranker on top, and attaching a human-readable natural-language explanation to every result. The system is served through a FastAPI backend, a React terminal-style frontend, and a Streamlit analytics dashboard, all operating on the MovieLens 100K/25M dataset.

2. Introduction

Recommendation systems are ubiquitous, yet transparency in these systems remains rare. Users who understand why a film is recommended are more likely to trust and engage with the suggestion. CINE-IQ was designed as a glass-box alternative to opaque collaborative filtering, giving analysts and end users full visibility into the signal breakdown behind each recommendation. The system supports two operating modes:

- Item-Item Mode:** Given a movie title or ID, return the most similar films using a hybrid of content, collaborative, and matrix-factorisation signals.
- Personal Mode:** Given a user ID, return personalised recommendations derived from their full rating history, enriched with TMDB cast, crew, and keyword metadata.

3. System Architecture

CINE-IQ operates on a decoupled microservices architecture designed to keep per-request latency under 100 ms after the initial warm-up phase. The pipeline flows through five distinct layers: query ingestion, model inference, ensemble fusion, sentiment re-ranking, and explainability generation.

3.1 Component Overview

Component	Technology	Role
React Frontend	React 18.3.1, Vite 5.4.0	Terminal-style query UI, scroll animations, fallback cache
FastAPI Backend	FastAPI 0.115.4, Uvicorn	Async routing, model orchestration, Pydantic validation
Content Model	TF-IDF + cosine similarity	Genre and tag vector similarity (item-item)
Collaborative Model	sklearn NMF	Latent item-factor dot-product scoring
Matrix Model	TruncatedSVD	Full user-item rating matrix factorisation
Sentiment Layer	vaderSentiment 3.3.2	Review-based score adjustment (±15%)
Explainability	LIME 0.2.0.1 + templates	Per-pair attribution and prose generation

Component	Technology	Role
Analytics Dashboard	Streamlit 1.39.0, Plotly	Model stats, score distribution, taste profiling
Experiment Tracking	MLflow 2.17.2	Offline training runs, metric logging

3.2 Decision Engine — Hybrid Ensemble

For a given seed movie, each model independently generates a ranked candidate pool of 200 films. Scores are normalised to [0, 1] and fused via a configurable weighted sum:

```
final_score = 0.40 × content + 0.40 × collaborative + 0.20 × matrix
```

All weights live in a single *config.py* file, enabling analysts to retune the blend without touching model code. The sentiment re-ranker then shifts each score by up to ±15% based on the aggregate VADER sentiment of associated user reviews. Because VADER is lexicon-based, this pass adds zero training overhead and operates in microseconds. A DistilBERT stub is included for replacement when higher classification accuracy is required.

4. Explainability Pipeline

Every recommendation includes two explanation artefacts that together give users and analysts a complete picture of the scoring decision:

- **Prose Reason:** A rule-based template fills in TF-IDF key terms to produce a sentence such as: "Shares themes of sci-fi dystopia and retrofuturism with Blade Runner (1982)." Highlighted terms are returned as HTML `<span class="hl">` elements, ready for the frontend to render directly.
- **Signal Breakdown:** A `signals` object reports the raw contribution from each model layer — content, collaborative, matrix, and sentiment — giving full auditability over why a score landed where it did.
- **LIME Attribution:** For deeper analysis, LIME feature attribution is available per recommendation pair, identifying which input features (genre tokens, rating patterns) drove the similarity score.

5. API Reference

5.1 Endpoint Surface

Meth od	Endpoint	Description
GET	/health	Service status, model readiness, dataset dimensions
GET	/search?q={query}	Fuzzy title search — returns movie metadata list
GET	/recommend?title={title}	Hybrid recommendations by title string
GET	/recommend?movie_id={id}	Hybrid recommendations by MovieLens integer ID
GET	/similar?movie_id={id}	Item-item similarity alias of /recommend
GET	/recommend/user/{id}	Personalised recs — strategy: content / collab / hybrid
GET	/explore/user/{id}	Taste profile: top genres, cast, directors, keywords

5.2 Recommendation Response Schema

Field	Type	Description
movie_id	int	MovieLens film identifier

Field	Type	Description
title	str	Film title with release year
score	float	Normalised ensemble score [0, 1]
score_pct	int	Score expressed as integer percentage
reason	str	HTML-safe natural-language explanation
signals.content	float	TF-IDF content model contribution
signals.collaborative	float	NMF collaborative model contribution
signals.matrix	float	SVD matrix factorisation contribution
signals.sentiment	float	VADER sentiment re-ranker adjustment

6. Data & Training

6.1 Dataset

CINE-IQ ships with a first-run downloader for two MovieLens variants. Both are loaded into in-memory pandas DataFrames at API startup. Optional TMDB metadata enrichment adds cast, crew, plot keywords, and production company data — required for personal recommendations and taste profiling.

Dataset	Movies	Ratings	Users	Approx. Size
MovieLens 100K (default)	1,682	100,000	943	~5 MB
MovieLens 25M (production)	62,423	25,000,000	162,541	~250 MB

6.2 Offline Training & Evaluation

Models can be trained offline via `scripts/train.py`, which fits all three model layers and logs hyperparameters, weights, and evaluation metrics to MLflow. Hit-rate@K validation (K = 5, 10, 20) is provided by `scripts/evaluate.py`. Fitted artefacts are persisted to the `artifacts/` directory and loaded at API startup, bypassing re-training on every launch. All model outputs are cached in-memory using `@lru_cache(maxsize=10,000)` to minimise repeated computation.

7. Testing

The test suite runs against a synthetic 30-movie fixture defined in `tests/conftest.py`, so all model and API tests execute without downloading the full MovieLens dataset. `pytest 8.3.3` and `httpx 0.27.2` are used for synchronous and asynchronous integration testing respectively.

Test Module	Coverage Area
<code>test_models.py</code>	Content, collaborative, matrix, and hybrid ensemble correctness
<code>test_sentiment_and_explain.py</code>	VADER scoring, LIME attribution, prose template generation
<code>test_api.py</code>	Full FastAPI integration via <code>httpx</code> async client
<code>test_personal_models.py</code>	Per-user NMF and content personalisation strategy
<code>test_loader.py</code>	Dataset parsing, column validation, TMDB enrichment

Test Module	Coverage Area
test_title_normalizer.py	Fuzzy title matching and normalisation edge cases

8. Conclusion

CINE-IQ demonstrates that recommendation transparency and production performance are not mutually exclusive. By fusing three complementary ML models with a sentiment post-processor and a dedicated explainability layer, the system delivers sub-100 ms recommendations alongside clear, auditable reasoning that users and analysts can act on.

The modular architecture — configurable ensemble weights, swappable sentiment backends, optional TMDB enrichment — makes it straightforward to extend for new datasets, domains, or model architectures. Future work includes full DistilBERT sentiment integration, automated A/B weight tuning via the MLflow interface, and a user feedback loop to close the recommendation cycle with implicit signal data.